Ejercicio Integrador - Sistema de Biblioteca

Nombre: _Erick Emmanuel Romero Mejía_

Instrucciones: Lee cuidadosamente cada sección y responde de forma clara y ordenada.

Parte 1: Análisis del sistema

Se desea desarrollar un sistema de biblioteca donde existen diferentes tipos de usuarios (alumnos y bibliotecarios). Los bibliotecarios registran libros y los alumnos solicitan préstamos. Todos los usuarios deben tener una descripción personalizada.

1. ¿Cuáles son las clases principales del sistema?

_Biblioteca, Prestamos, Gestión Prestamos, Usuario, Libro, Bibliotecario y Alumno

2. ¿Cuál es la clase base?

_Biblioteca

3. ¿Qué clases heredan de otras?

_Bibliotecario, Alumno, Gestión Prestamos

4. ¿Qué clase debería ser abstracta?

_Usuario

Parte 2: Herencia y Tipos Abstractos

Completa e implementa las clases:

```
class Usuario(ABC):
    def __init__(self, nombre):
        self._nombre = nombre

    @abstractmethod
    def descripcion(self):
        pass
```

Instrucciones:
- Crear clase Alumno
- Crear clase Bibliotecario
- Implementar descripcion()

Evidencia:

alumno.py

```
from modelos.usuario import Usuario

class Alumno(Usuario):

        def __init__(self,nombre,identificador,tipo_usuario):

                super().__init__(nombre,identificador,tipo_usuario)

        def pedirLibro(self):

                print("El alumno solicita un libro")


from modelos.usuario import Usuario

from modelos.biblioteca import Biblioteca

class Bibliotecario(Usuario):

        def __init__(self,nombre,identificador,area):

                super().__init__(nombre,identificador,"Bibliotecario")

                self._area = area


        def obtener_area(self):

                return self._area

        def registrar_libro(self,biblioteca,libro):

                biblioteca.agregar_libros(libro)

                print(f"{self.getNombre()} registro el libro {libro.getTitulo()}")
```

Parte 3: Encapsulamiento

Modifica la clase Libro:

```
class Libro:
    def __init__(self, titulo):
```

```
        self.titulo = titulo
```

Instrucciones:
- Convertir atributo en protegido
- Crear método getter

Evidencia:

```
class Libro:

        def __init__(self, titulo):
                self._titulo = titulo

        get_titulo(self):

                return self._titulo
```

Parte 4: Lógica del sistema

Completa el método:

```
class GestorPrestamos:
    def realizar_prestamo(self, libro, usuario):
        # completar
```

Requisitos:
- Validar disponibilidad
- Crear préstamo
- Guardarlo en lista

Evidencia:

```
from modelos.libro import Libro

from modelos.prestamo import Prestamo

class GestorPrestamos:

        def __init__(self):

        self.prestamos = []
```

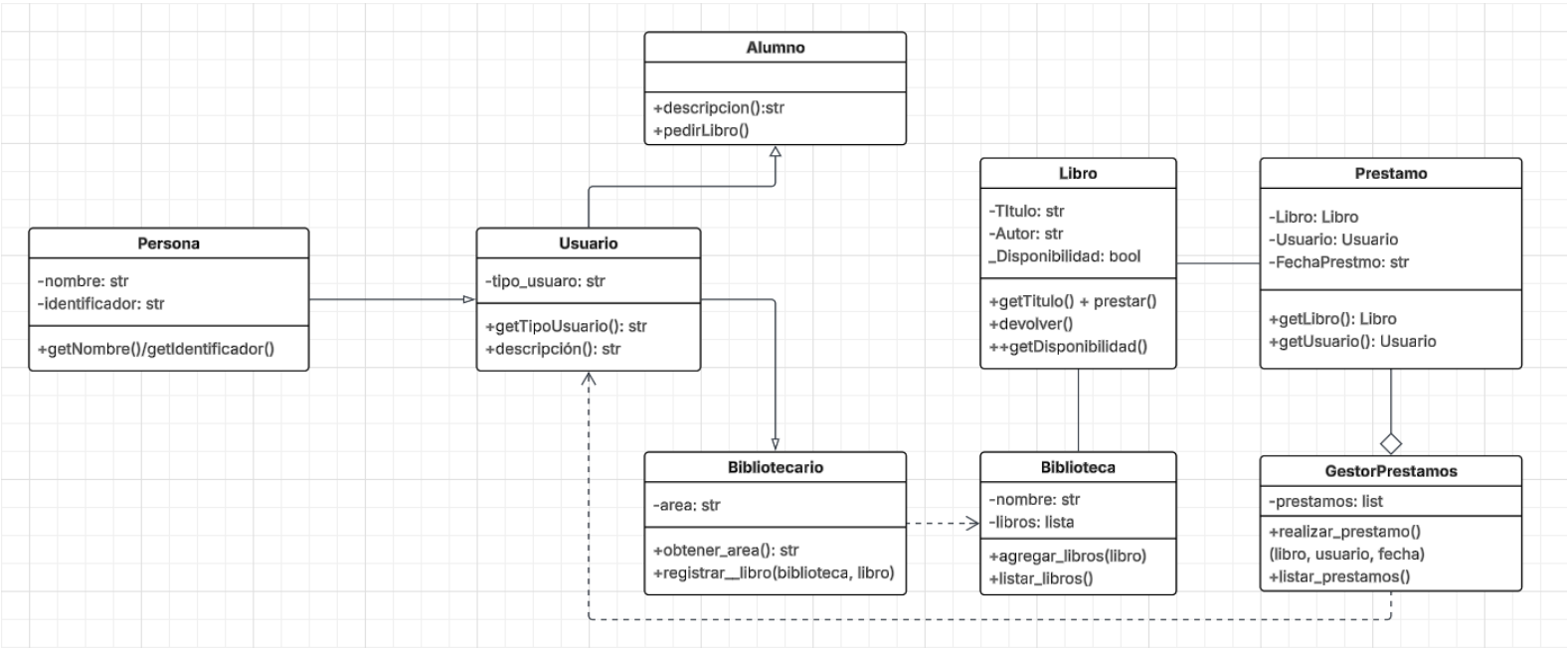
```
        def realizar_prestamo(self,libro,usuario,fecha):
        if libro.prestar():
                prestamo = Prestamo(libro,usuario,fecha)
                self.prestamos.append(prestamo)
                return "Prestamo realizado correctamente"
        return "El libro no esta disponible"
        def listar_prestamos(self):
                for prestamo in self.prestamos:
                        print(prestamo)


class Prestamo:
        def __init__(self,libro,usuario,fechaprestamo):
                self.__Libro = libro
                self.__Usuario = usuario
                self.__FechaPrestamo = fechaprestamo
        def getUsuario(self):
                return self.__Usuario
         def getLibro(self):
                return self.__Libro
        def getFechaPrestamo(self):
                return self.__FechaPrestamo
        def __str__(self):
                return f"{self.__Usuario} -- {self.__Libro}"
```

Parte 5: UML

Dibuja el diagrama de clases incluyendo:
- Persona
- Usuario (abstracto)
- Alumno
- Bibliotecario
- Libro
- Prestamo
- GestorPrestamos

Incluir:
- Herencia
- Asociación
- Composición

Evidencia:



Parte 6: Identificación de errores

Analiza el código:

```
usuario1 = Usuario('Ana')
```

¿Qué error tiene?

En si la clase Usuario no se encarga de crear el usuario, quien registra los datos y con quien se instancia es con alumno o bibliotecario, es decir, usuario es una clase abstracta, lo que impide que pueda instanciarse el usuario de forma directa

Parte 7: Ejecución del sistema

Completa el main.py:

- Crear alumno
- Crear libro
- Realizar préstamo
- Mostrar resultado

Evidencia:

```
from modelos.libro import Libro

from modelos.usuario import Usuario

from servicios.gestor_prestamos import GestorPrestamos

from modelos.biblioteca import Biblioteca

from modelos.bibliotecario import Bibliotecario

from modelos.alumno import Alumno

biblioteca1 = Biblioteca("Biblioteca Central")

bibliotecario1 = Bibliotecario("Carlos","B001","IA")

libro1 = Libro("Peppa Pig","Bucanero","12345")

libro2 = Libro("Teoria de Evolucion del Ser Humano","Jorge el Curioso","12345")

bibliotecario1.registrar_libro(biblioteca1,libro1)

bibliotecario1.registrar_libro(biblioteca1,libro2)

biblioteca1.listar_libros()

alumno1 = Alumno("Erick","2214991","Alumno")
```

```
gestor = GestorPrestamos()

gestor.realizar_prestamo(libro1,alumno1,"18-04-2026")

gestor.listar_prestamos()
```